EE789:Algorithmic Design of Digital Systems
Assignment 2 : Problem 2 Report

Name of student : Prathmesh Baral
Roll no : 23B1260

April 12, 2026

---

1 Problem Statement

For each optimization , incorporate the optimization in your algorithm and report the improvement in performance for N=64

2 Base Implementation

In the base implementation, matrix multiplication is carried out in a straightforward and sequential manner without applying any optimizations. All matrices (A, B, and $RESULT$) are stored as two-dimensional arrays of 8-bit integers.

The computation follows the standard triple-nested loop structure. For each element $RESULT[I][J]$, a dot-product is computed between the I^{th} row of matrix A and the J^{th} column of matrix B. This functionality is implemented in the `dotp` module.

Within the `dotp` module, the accumulation is performed iteratively:

- A single multiplication `A[I][K] * B[K][J]` is computed in each iteration.
- The loop variable K is incremented by 1, resulting in N iterations per dot-product.
- A single accumulator (`SUM`) is used to store intermediate results.

The top-level `mmul` module iterates over all indices (I, J) to compute the complete $N \times N$ result matrix. The execution time is measured using a global counter, which tracks the number of clock cycles (ticks) required for the computation.

Since only one multiplication is performed per cycle (ignoring pipeline effects), this implementation serves as a baseline for evaluating the effectiveness of subsequent optimizations.

2.1 Tick count : 2,670,785

```
RESULT[63][60] = 123
RESULT[63][61] = 124
RESULT[63][62] = 125
RESULT[63][63] = 126
[INFO] Results fetched successfully.
[INFO] Computation finished. Ticks = 2670785

---------------- MATRIX A ----------------
   0   1   2   3   4   5   6   7   8   9  10  11  12  13
42  43  44  45  46  47  48  49  50  51  52  53  54  55
```

Figure 1: Tick count for base implementation

3 Optimization 1: Loop Unrolling (Factor of 2)

In this optimization, the inner dot-product computation is improved by performing two multiplications in parallel within each iteration. Instead of processing a single element per loop iteration, the loop is unrolled by a factor of 2, allowing two independent multiply-accumulate operations to be executed simultaneously.

This is implemented in the `dotp_unroll_2` module, where two accumulators (`SUM0` and `SUM1`) are maintained. At each step, the algorithm computes:

- `A[I][K] * B[K][J]`
- `A[I][K+1] * B[K+1][J]`

and accumulates them separately before combining the results at the end.

The loop index is incremented by 2 in each iteration, effectively halving the number of iterations required for the dot-product computation. This improves throughput by exploiting instruction-level parallelism and better utilization of the pipeline.

Overall, this optimization reduces the total execution time compared to the base implementation by decreasing the number of loop iterations and enabling concurrent arithmetic operations.

3.1 Tick count : 1,749,185

3.2 You can refer the code for opt 1 at section 7

3.3 Worked out examples can be referred at section 10

The code bundle is stored in AA_`code_bundles/Optimization1`.
The testbench and corresponding log files are available in `Testing_files_outputs/Optimization1`.

```
RESULT[63][61] = 124
RESULT[63][62] = 125
RESULT[63][63] = 126
[INFO] Results fetched successfully.
[INFO] Computation finished. Ticks = 1749185

---------------- MATRIX A ----------------
   0   1   2   3   4   5   6   7   8   9  10  11  12  13  14
47  48  49  50  51  52  53  54  55  56  57  58  59  60  61  62
   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15
48  49  50  51  52  53  54  55  56  57  58  59  60  61  62  63
```

Figure 2: Tick count for optimization 1

4 Optimization 2: Word-parallelism

In this optimization, the data layout and computation strategy are modified to improve memory bandwidth utilization and exploit parallelism. Matrix A is stored in row-major format, while matrix B is stored in column-major format.

Instead of storing individual 8-bit elements, both matrices are packed into 64-bit words, where each word contains 8 consecutive elements. Thus, each access retrieves a block of 8 elements at once.

The dot-product computation is redesigned using a `dot8` module, which processes 8 pairs of elements in parallel. Each 64-bit input is split into eight 8-bit values, followed by:

- 8 parallel multiplications
- A tree-structured addition to accumulate results efficiently

In the updated `dotp` module, the loop now iterates over packed elements (stride of 8), reducing the number of iterations from N to $N/8$. In each iteration, a single call to `dot8` computes the contribution of 8 elements simultaneously.

This optimization significantly improves performance by:

- Reducing memory accesses through data packing
- Exploiting data-level parallelism (8 multiplications at once)
- Lowering loop overhead due to fewer iterations

4.1 Tick count : 647,361

4.2 You can refer the code for opt 2 at section 8

4.3 Worked out examples can be referred at section 11

The code bundle is stored in AA_code_bundles/Optimization2.
The testbench and corresponding log files are available in Testing_files_outputs/Optimization2.

```
RESULT[63][60] = 123
RESULT[63][61] = 124
RESULT[63][62] = 125
RESULT[63][63] = 126
[INFO] Results fetched successfully.
[INFO] Computation finished. Ticks = 647361

---------------- MATRIX A ----------------
   0   1   2   3   4   5   6   7   8   9  10  11  12
42  43  44  45  46  47  48  49  50  51  52  53  54  55
   1   2   3   4   5   6   7   8   9  10  11  12  13
```

Figure 3: Tick count for optimization 2

5 Optimization 3: Blocking and Tiling

In this optimization, the matrix multiplication is improved using a blocking (tiling) technique. The original matrices are divided into four sub-matrices of size $\frac{N}{2} \times \frac{N}{2}$, resulting in partitions A_0, A_1, A_2, A_3 and B_0, B_1, B_2, B_3.

Each element of the result matrix is computed using these smaller blocks. Instead of computing one output element at a time, four corresponding elements are computed simultaneously:

- $R[I][J] = \sum_{k=0}^{\frac{N}{2}-1} \left(A_0[I][k] \cdot B_0[k][J] + A_1[I][k] \cdot B_2[k][J]\right)$
- $R[I][J+N/2] = \sum_{k=0}^{\frac{N}{2}-1} \left(A_0[I][k] \cdot B_1[k][J] + A_1[I][k] \cdot B_3[k][J]\right)$
- $R[I+N/2][J] = \sum_{k=0}^{\frac{N}{2}-1} \left(A_2[I][k] \cdot B_0[k][J] + A_3[I][k] \cdot B_2[k][J]\right)$
- $R[I+N/2][J+N/2] = \sum_{k=0}^{\frac{N}{2}-1} \left(A_2[I][k] \cdot B_1[k][J] + A_3[I][k] \cdot B_3[k][J]\right)$

This is implemented in the `mmul` module, where four dot-product modules (`dotp0`, `dotp1`, `dotp2`, `dotp3`) are invoked in parallel for each (I, J) index pair.

Each dot-product module operates on smaller sub-matrices and accumulates partial results from two blocks, improving data locality and enabling parallel computation within each iteration. :contentReference[oaicite:0]index=0

The loop bounds are also reduced from N to $N/2$, which decreases the total number of iterations. Overall, this optimization enhances performance by:

- Improving cache/memory locality through smaller blocks
- Enabling parallel computation of multiple output elements
- Reducing loop overhead due to smaller iteration space

5.1 Tick count : 442,465

5.2 You can refer the code for opt 3 at section 9

5.3 Worked out examples can be referred at section 12

The code bundle is stored in AA_`code_bundles/Optimization3`.
The testbench and corresponding log files are available in `Testing_files_outputs/Optimization3`.

```
RESULT[63][60] = 123
RESULT[63][61] = 124
RESULT[63][62] = 125
RESULT[63][63] = 126
[INFO] Results fetched successfully.
[INFO] Computation finished. Ticks = 442465

---------------- MATRIX A ----------------
```

Figure 4: Tick count for optimization 3

6 Performance Comparison and Conclusion

The performance of matrix multiplication was evaluated for the base implementation and three successive optimizations. The execution time was measured in terms of clock ticks for $N = 64$.

Implementation	Ticks	Speedup (w.r.t Base)
Base Implementation	2,670,785	1.00×
Optimization 1 (Unroll ×2)	1,749,185	1.53×
Optimization 2 (Packing + dot8)	647,361	4.13×
Optimization 3 (Blocking/Tiling)	442,465	6.03×

- From the results, it is evident that each optimization progressively improves performance.
- Optimization 1 reduces execution time by exploiting instruction-level parallelism through loop unrolling, decreasing the number of iterations in the inner loop.
- Optimization 2 provides a significant improvement by packing data and performing 8 multiplications in parallel using the `dot8` module. This reduces both memory access overhead and loop iterations.
- Optimization 3 achieves the best performance by applying blocking and tiling. By dividing matrices into smaller sub-blocks, it improves data locality and enables simultaneous computation of multiple output elements, leading to better utilization of computational resources.
- Overall, the final optimized implementation achieves a speedup of more than 6× compared to the base implementation. This demonstrates the effectiveness of combining parallelism, data layout optimization, and blocking techniques in improving matrix multiplication performance.

7 Optimized Implementation 1 (Unroll Factor = 2) : Code

```
$parameter N        64
$pipe GLOBAL_COUNTER: $uint<32> $signal
$storage A B RESULT: $array [N][N] $of $int<8>

// to initialize the matrices..
$module [init_A_entry]
        $in (I J: $uint<32> V: $int<8>) $out () $is
{
        A[I][J] := ($bitcast ($int<8>) V)
}

$module [init_B_entry]
        $in (I J: $uint<32> V: $int<8>) $out () $is
{
        B[I][J] := ($bitcast ($int<8>) V)
}

// to read the matrices
$module [get_A_entry]
        $in (I J: $uint<32>) $out(V: $int<8>) $is
{
        V := A[I][J]
}

$module [get_B_entry]
        $in (I J: $uint<32>) $out(V: $int<8>) $is
{
        V := B[I][J]
}

$module [get_result_entry]
        $in (I J: $uint<32>) $out(V: $int<8>) $is
{
        V := RESULT[I][J]
}

// Counter to measure time..
$module [counter_daemon] $in () $out () $is
{
        $branchblock[loop] {
                $dopipeline $depth 7 $fullrate
                $merge $entry $loopback
                        $phi cval := $zero<32> $on $entry n_cval $on $loopbac
                $endmerge
                $volatile n_cval := (cval + 1)
                GLOBAL_COUNTER   := n_cval
                $while 1
        }
}
```

```
$module [mmul] $in () $out (nticks: $uint<32>) $is
{
        start_time := GLOBAL_COUNTER
        $branchblock[bb] {
              $merge $entry I_loopback
                  $phi I := $zero<8> $on $entry nI $on I_loopback
              $endmerge
              $volatile nI := (I + 1)
              $merge $entry J_loopback
                  $phi J := $zero<8> $on $entry nJ $on J_loopback
              $endmerge
              $volatile nJ := (J + 1)
              RESULT[I][J] := ($call dotp_unroll_2 (I J))
              $if (J < {N - 1}) $then $place [J_loopback] $endif
              $if (I < {N - 1}) $then $place [I_loopback] $endif
        }
        nticks := (GLOBAL_COUNTER - start_time)
}

$module [dotp_unroll_2] $in (I J: $uint<8>) $out (R: $int<8>) $is
{
      $branchblock[dotp] {
         $dopipeline $depth 7 $fullrate
         $merge $entry $loopback
              $phi SUM0 := ($bitcast ($int<8>) 0)  $on $entry nSUM0 $on $loop
              $phi SUM1 := ($bitcast ($int<8>) 0)  $on $entry nSUM1 $on $loop
              $phi K    := $zero<8> $on $entry nK $on $loopback
         $endmerge
         $volatile nK := (K + 2)
         $volatile K_continue := (K < {N-2})
         nSUM0 := (SUM0 + (A[I][K] * B[K][J]))
         nSUM1 := (SUM1 + (A[I][(K+1)] * B[(K+1)][J]))
         $while K_continue
        } (nSUM0 => S0 nSUM1 => S1)
        R := (S0 + S1)
}
```

8 Optimized Implementation 2 (Word-parallelism) : Code

```
$parameter N       64
$pipe GLOBAL_COUNTER: $uint<32> $signal
$storage RESULT : $array [N][N] $of $int<8>
$storage A B : $array[N][{N/8}] $of $uint<64>
// to initialize the matrices..
$module [init_A_entry]
        $in (I J: $uint<32> V: $uint<64>) $out () $is
{
        A[I][J] := ($bitcast ($uint<64>) V)
}

$module [init_B_entry]
```

```
        $in (I J: $uint<32> V: $uint<64>) $out () $is
{
        B[I][J] := ($bitcast ($uint<64>) V)
}

$module [get_result_entry]
        $in (I J: $uint<32>) $out(V: $int<8>) $is
{
        V := RESULT[I][J]
}

// Counter to measure time..
$module [counter_daemon] $in () $out () $is
{
        $branchblock[loop] {
                $dopipeline $depth 7 $fullrate
                $merge $entry $loopback
                        $phi cval := $zero<32> $on $entry n_cval $on $loopbac
                $endmerge
                $volatile n_cval := (cval + 1)
                GLOBAL_COUNTER   := n_cval
                $while 1
        }
}

$module [mmul] $in () $out (nticks: $uint<32>) $is
{
        start_time := GLOBAL_COUNTER
        $branchblock[bb] {
              $merge $entry I_loopback
                  $phi I := $zero<8> $on $entry nI $on I_loopback
              $endmerge
              $volatile nI := (I + 1)

              $merge $entry J_loopback
                  $phi J := $zero<8> $on $entry nJ $on J_loopback
              $endmerge
              $volatile nJ := (J + 1)

              RESULT[I][J] := ($call dotp (I J))

              $if (J < {N - 1}) $then $place [J_loopback] $endif
              $if (I < {N - 1}) $then $place [I_loopback] $endif
        }
        nticks := (GLOBAL_COUNTER - start_time)
}

$module [dotp] $in (I J: $uint<8>) $out (R: $int<8>) $is
{
      $branchblock[dotp] {
         $dopipeline $depth 7 $fullrate
         $merge $entry $loopback
```

```
            $phi SUM := ($bitcast ($int<8>) 0)  $on $entry nSUM $on $loopba
            $phi K   := $zero<8> $on $entry nK $on $loopback
         $endmerge
        $volatile nK := (K + 1)
        $volatile K_continue := (K < {{N/8}-1})
        nSUM := (SUM + ($call dot8 (A[I][K] B[J][K])))
         $while K_continue
        } (nSUM => nSUM)

        R := nSUM
}

$pipeline $depth 7 $fullrate $operator
$module[dot8] $in (A B: $uint<64>) $out (R: $int<8>)
$is
{
        // Split into 8 elements (each 8-bit)
        $volatile $split (A 8 8 8 8 8 8 8 8) (A0 A1 A2 A3 A4 A5 A6 A7)
        $volatile $split (B 8 8 8 8 8 8 8 8) (B0 B1 B2 B3 B4 B5 B6 B7)
        // Convert to signed
        $volatile iA0 := ($bitcast ($int<8>) A0)
        $volatile iA1 := ($bitcast ($int<8>) A1)
        $volatile iA2 := ($bitcast ($int<8>) A2)
        $volatile iA3 := ($bitcast ($int<8>) A3)
        $volatile iA4 := ($bitcast ($int<8>) A4)
        $volatile iA5 := ($bitcast ($int<8>) A5)
        $volatile iA6 := ($bitcast ($int<8>) A6)
        $volatile iA7 := ($bitcast ($int<8>) A7)
        $volatile iB0 := ($bitcast ($int<8>) B0)
        $volatile iB1 := ($bitcast ($int<8>) B1)
        $volatile iB2 := ($bitcast ($int<8>) B2)
        $volatile iB3 := ($bitcast ($int<8>) B3)
        $volatile iB4 := ($bitcast ($int<8>) B4)
        $volatile iB5 := ($bitcast ($int<8>) B5)
        $volatile iB6 := ($bitcast ($int<8>) B6)
        $volatile iB7 := ($bitcast ($int<8>) B7)
        // 8 parallel multiplications
        P0 := (iA0 * iB0)
        P1 := (iA1 * iB1)
        P2 := (iA2 * iB2)
        P3 := (iA3 * iB3)
        P4 := (iA4 * iB4)
        P5 := (iA5 * iB5)
        P6 := (iA6 * iB6)
        P7 := (iA7 * iB7)
        // Adder tree (log depth reduction)
        S0 := (P0 + P1)
        S1 := (P2 + P3)
        S2 := (P4 + P5)
        S3 := (P6 + P7)
        T0 := (S0 + S1)
        T1 := (S2 + S3)
```

```
        R := (T0 + T1)
}
```

9 Optimized Implementation 2 (Blocking and Tiling) : Code

```
$parameter N        64
$pipe GLOBAL_COUNTER: $uint<32> $signal
$storage A0 A1 A2 A3 : $array [{N/2}][{N/2}] $of $int<8>
$storage B0 B1 B2 B3 : $array [{N/2}][{N/2}] $of $int<8>
$storage R : $array [N][N] $of $int<8>

// to initialize the matrices..
$module [init_A0_entry]
        $in (I J: $uint<32> V: $int<8>) $out () $is
{
        A0[I][J] := ($bitcast ($int<8>) V)
}

$module [init_A1_entry]
        $in (I J: $uint<32> V: $int<8>) $out () $is
{
        A1[I][J] := ($bitcast ($int<8>) V)
}

$module [init_A2_entry]
        $in (I J: $uint<32> V: $int<8>) $out () $is
{
        A2[I][J] := ($bitcast ($int<8>) V)
}

$module [init_A3_entry]
        $in (I J: $uint<32> V: $int<8>) $out () $is
{
        A3[I][J] := ($bitcast ($int<8>) V)
}

$module [init_B0_entry]
        $in (I J: $uint<32> V: $int<8>) $out () $is
{
        B0[I][J] := ($bitcast ($int<8>) V)
}

$module [init_B1_entry]
        $in (I J: $uint<32> V: $int<8>) $out () $is
{
        B1[I][J] := ($bitcast ($int<8>) V)
}

$module [init_B2_entry]
        $in (I J: $uint<32> V: $int<8>) $out () $is
{
```

```
        B2[I][J] := ($bitcast ($int<8>) V)
}

$module [init_B3_entry]
        $in (I J: $uint<32> V: $int<8>) $out () $is
{
        B3[I][J] := ($bitcast ($int<8>) V)
}

// to read the matrices
$module [get_A0_entry]
        $in (I J: $uint<32>) $out(V: $int<8>) $is
{
        V := A0[I][J]
}

$module [get_A1_entry]
        $in (I J: $uint<32>) $out(V: $int<8>) $is
{
        V := A1[I][J]
}

$module [get_A2_entry]
        $in (I J: $uint<32>) $out(V: $int<8>) $is
{
        V := A2[I][J]
}

$module [get_A3_entry]
        $in (I J: $uint<32>) $out(V: $int<8>) $is
{
        V := A3[I][J]
}

$module [get_B0_entry]
        $in (I J: $uint<32>) $out(V: $int<8>) $is
{
        V := B0[I][J]
}

$module [get_B1_entry]
        $in (I J: $uint<32>) $out(V: $int<8>) $is
{
        V := B1[I][J]
}

$module [get_B2_entry]
        $in (I J: $uint<32>) $out(V: $int<8>) $is
{
        V := B2[I][J]
}
```

```
$module [get_B3_entry]
        $in (I J: $uint<32>) $out(V: $int<8>) $is
{
        V := B3[I][J]
}

$module [get_result_entry]
        $in (I J: $uint<32>) $out(V: $int<8>) $is
{
        V := R[I][J]
}

// Counter to measure time..
$module [counter_daemon] $in () $out () $is
{
        $branchblock[loop] {
                $dopipeline $depth 7 $fullrate
                $merge $entry $loopback
                        $phi cval := $zero<32> $on $entry n_cval $on $loopbac
                $endmerge
                $volatile n_cval := (cval + 1)
                GLOBAL_COUNTER   := n_cval

                $while 1
        }
}


$module [mmul] $in () $out (nticks: $uint<32>) $is
{
        start_time := GLOBAL_COUNTER
        $branchblock[bb] {
              $merge $entry I_loopback
                  $phi I := $zero<8> $on $entry nI $on I_loopback
              $endmerge
              $volatile nI := (I + 1)

              $merge $entry J_loopback
                  $phi J := $zero<8> $on $entry nJ $on J_loopback
              $endmerge
              $volatile nJ := (J + 1)

              i2 := (I + (N/2))
              j2 := (J + (N/2))

              R[I][J] := ($call dotp0 (I J))
              R[I][j2] := ($call dotp1 (I J))
              R[i2][J] := ($call dotp2 (I J))
              R[i2][j2] := ($call dotp3 (I J))


              $if (J < {{N/2} - 1}) $then $place [J_loopback] $endif
```

```
            $if (I < {{N/2} - 1}) $then $place [I_loopback] $endif
        }
        nticks := (GLOBAL_COUNTER - start_time)
}

$module [dotp0] $in (I J: $uint<8>) $out (R: $int<8>) $is
{
      $branchblock[dotp0] {
         $dopipeline $depth 7 $fullrate

         $merge $entry $loopback
              $phi SUM0 := ($bitcast ($int<8>) 0)  $on $entry nSUM0 $on $loop
              $phi SUM1 := ($bitcast ($int<8>) 0)  $on $entry nSUM1 $on $loop
              $phi K    := $zero<8> $on $entry nK $on $loopback
         $endmerge

         $volatile nK := (K + 1)
         $volatile K_continue := (K < {{N/2}-1})

         nSUM0 := (SUM0 + (A0[I][K] * B0[K][J]))
         nSUM1 := (SUM1 + (A1[I][K] * B2[K][J]))
         $while K_continue
        } (nSUM0 => nSUM0 nSUM1 => nSUM1)

        R := (nSUM0 + nSUM1)
}

$module [dotp1] $in (I J: $uint<8>) $out (R: $int<8>) $is
{
      $branchblock[dotp1] {
         $dopipeline $depth 7 $fullrate

         $merge $entry $loopback
              $phi SUM0 := ($bitcast ($int<8>) 0)  $on $entry nSUM0 $on $loop
              $phi SUM1 := ($bitcast ($int<8>) 0)  $on $entry nSUM1 $on $loop
              $phi K    := $zero<8> $on $entry nK $on $loopback
         $endmerge

         $volatile nK := (K + 1)
         $volatile K_continue := (K < {{N/2}-1})

         nSUM0 := (SUM0 + (A0[I][K] * B1[K][J]))
         nSUM1 := (SUM1 + (A1[I][K] * B3[K][J]))
         $while K_continue
        } (nSUM0 => nSUM0 nSUM1 => nSUM1)

        R := (nSUM0 + nSUM1)
}

$module [dotp2] $in (I J: $uint<8>) $out (R: $int<8>) $is
{
      $branchblock[dotp2] {
```

```
        $dopipeline $depth 7 $fullrate

        $merge $entry $loopback
             $phi SUM0 := ($bitcast ($int<8>) 0)  $on $entry nSUM0 $on $loop
             $phi SUM1 := ($bitcast ($int<8>) 0)  $on $entry nSUM1 $on $loop
             $phi K    := $zero<8> $on $entry nK $on $loopback
        $endmerge

        $volatile nK := (K + 1)
        $volatile K_continue := (K < {{N/2}-1})

        nSUM0 := (SUM0 + (A2[I][K] * B0[K][J]))
        nSUM1 := (SUM1 + (A3[I][K] * B2[K][J]))
        $while K_continue
       } (nSUM0 => nSUM0 nSUM1 => nSUM1)

       R := (nSUM0 + nSUM1)
}

$module [dotp3] $in (I J: $uint<8>) $out (R: $int<8>) $is
{
      $branchblock[dotp3] {
        $dopipeline $depth 7 $fullrate

        $merge $entry $loopback
             $phi SUM0 := ($bitcast ($int<8>) 0)  $on $entry nSUM0 $on $loop
             $phi SUM1 := ($bitcast ($int<8>) 0)  $on $entry nSUM1 $on $loop
             $phi K    := $zero<8> $on $entry nK $on $loopback
        $endmerge

        $volatile nK := (K + 1)
        $volatile K_continue := (K < {{N/2}-1})

        nSUM0 := (SUM0 + (A2[I][K] * B1[K][J]))
        nSUM1 := (SUM1 + (A3[I][K] * B3[K][J]))
        $while K_continue
       } (nSUM0 => nSUM0 nSUM1 => nSUM1)

       R := (nSUM0 + nSUM1)
}
```

10 Optimized Implementation 1 (Unroll Factor = 2) : Working Screenshot

10.1 Test 1 : A x I

A matrix is such that $A[x][y] = x + y$ and I is identity matrix which results as output A

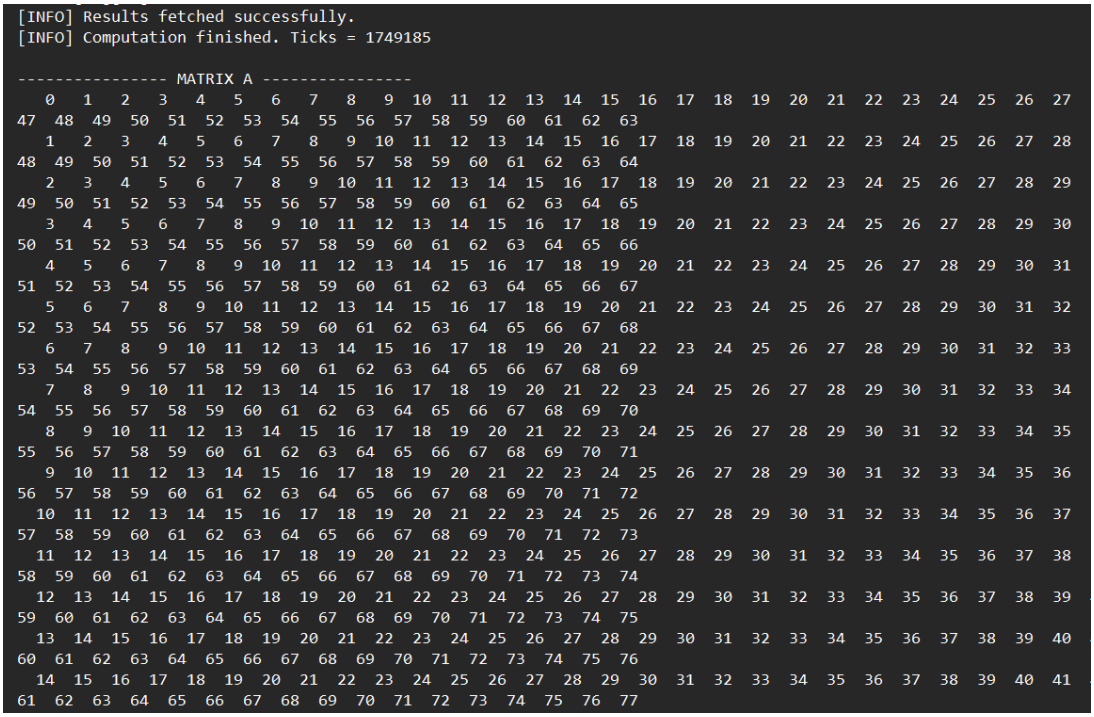

Figure 5: matrix A

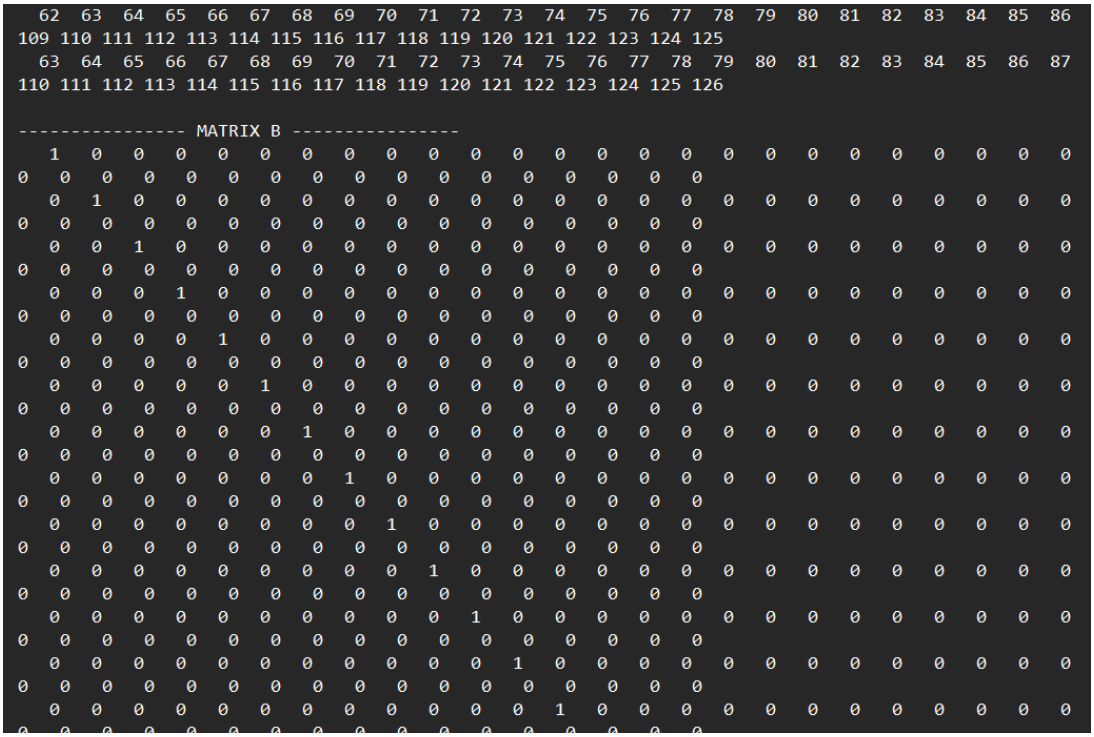

Figure 6: matrix B

```
0   0   0   0   0   0   0   0   0   0   0   0   0   0   1   0   0
   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   1   0
   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   1

-------------- RESULT MATRIX --------------
   0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19  20  21
47  48  49  50  51  52  53  54  55  56  57  58  59  60  61  62  63
   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19  20  21  22
48  49  50  51  52  53  54  55  56  57  58  59  60  61  62  63  64
   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19  20  21  22  23
49  50  51  52  53  54  55  56  57  58  59  60  61  62  63  64  65
   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19  20  21  22  23  24
50  51  52  53  54  55  56  57  58  59  60  61  62  63  64  65  66
   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19  20  21  22  23  24  25
51  52  53  54  55  56  57  58  59  60  61  62  63  64  65  66  67
   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26
52  53  54  55  56  57  58  59  60  61  62  63  64  65  66  67  68
   6   7   8   9  10  11  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26  27
53  54  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69
   7   8   9  10  11  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26  27  28
54  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70
   8   9  10  11  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26  27  28  29
55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71
   9  10  11  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26  27  28  29  30
56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
  10  11  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31
57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72  73
  11  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31  32
58  59  60  61  62  63  64  65  66  67  68  69  70  71  72  73  74
  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33
59  60  61  62  63  64  65  66  67  68  69  70  71  72  73  74  75
```

Figure 7: result matrix

10.2 Test 2 : A x B

A matrix is such that $A[x][y] = x \; mod \; 4$ and $B[x][y] = y \; mod \; 4$

```
[INFO] Results fetched successfully.
[INFO] Computation finished. Ticks = 1749185

---------------- MATRIX A ----------------
   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1
1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1
   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2
2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2
   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3
3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3
   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1
1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1
```

Figure 8: matrix A

```
---------------- MATRIX B ----------------
   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0
2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3
   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0
2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3
   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0
2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3
   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0
2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3
   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0
2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3
   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0
2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3
   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0
2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3
   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0   1   2   3   0
```

Figure 9: matrix B

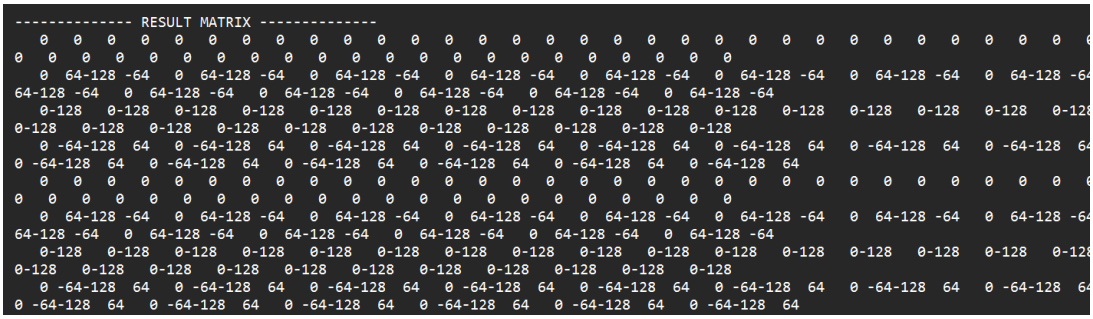

Figure 10: result matrix

Note: The computed values are clipped to 8-bit signed representation. Hence, overflow leads to wrapping, which results in negative values appearing in the output.

10.3 Test 3 : A x B

A matrix is such that $A[x][y] = x \ mod \ 4$ and $B[x][y] = x \ mod \ 4$

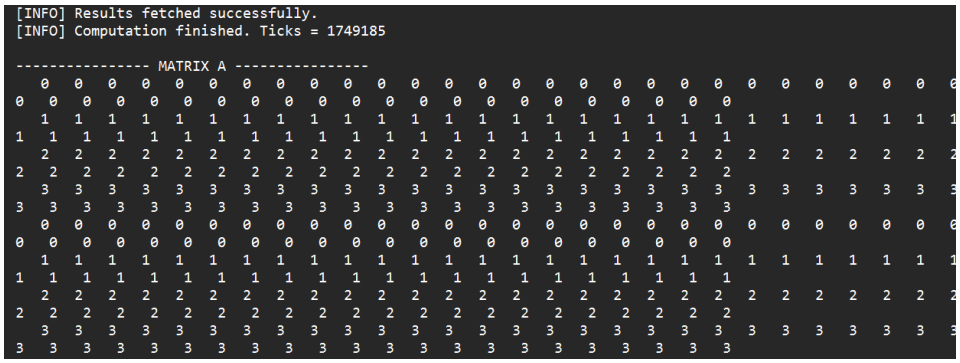

Figure 11: matrix A

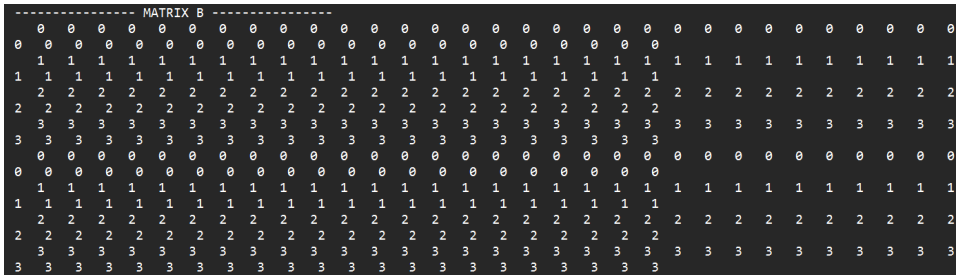

Figure 12: matrix B

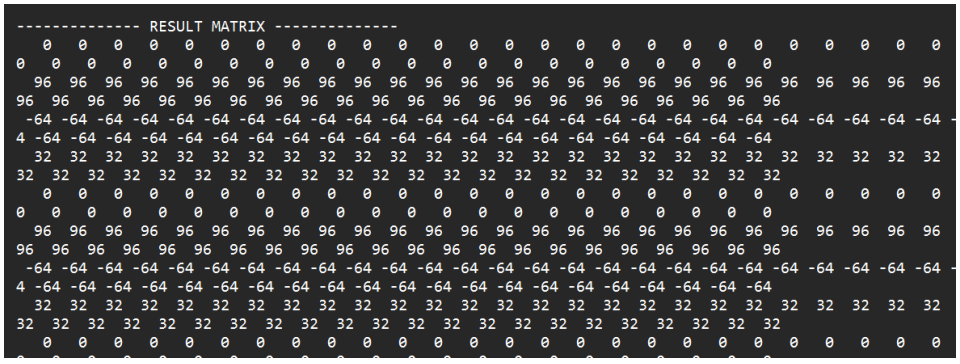

Figure 13: result matrix

11 Optimized Implementation 2 (Word-parallelism) : Working Screenshot

11.1 Test 1 : A x I

A matrix is such that $A[x][y] = x + y$ and I is identity matrix which results as output A

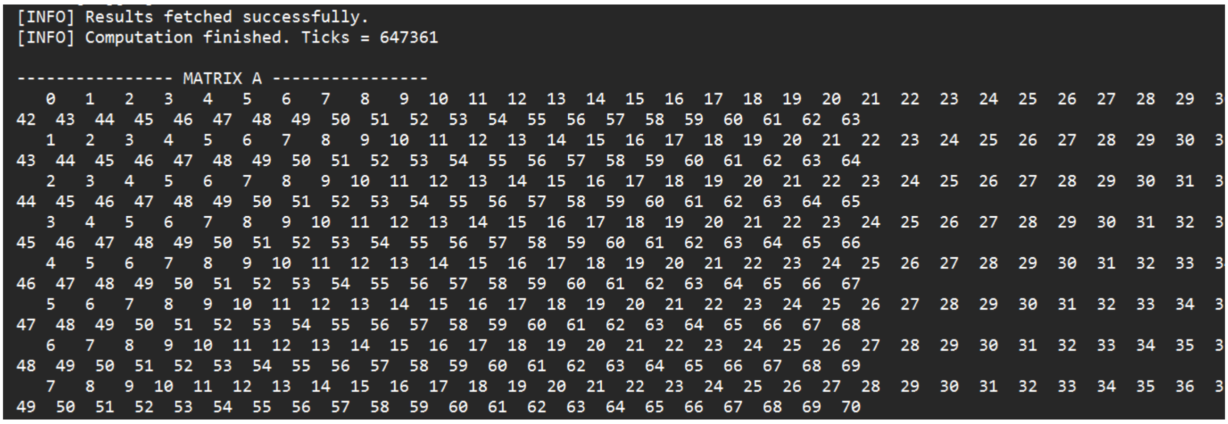

Figure 14: matrix A

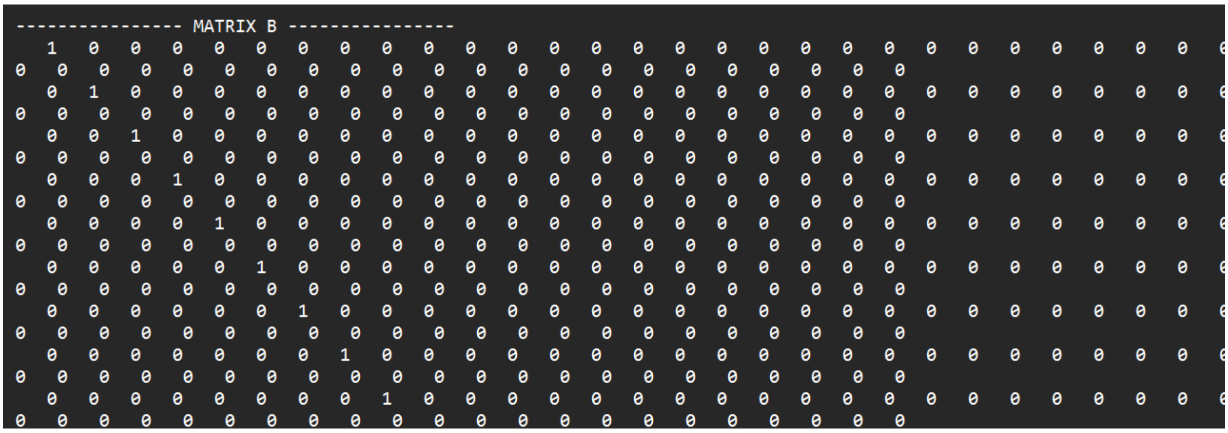

Figure 15: matrix B

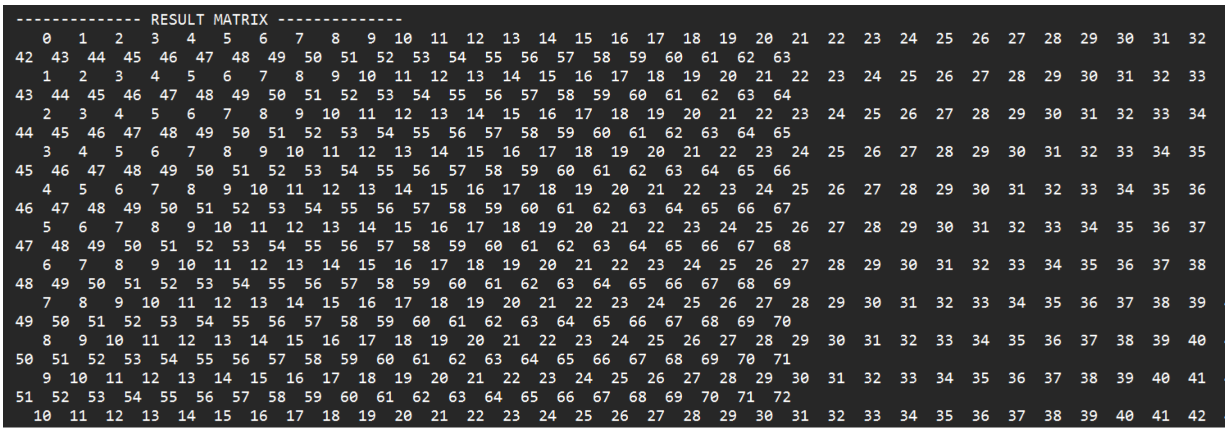

Figure 16: result matrix

11.2 Test 2 : A x B

A matrix is such that $A[x][y] = x \ mod \ 4$ and $B[x][y] = y \ mod \ 4$

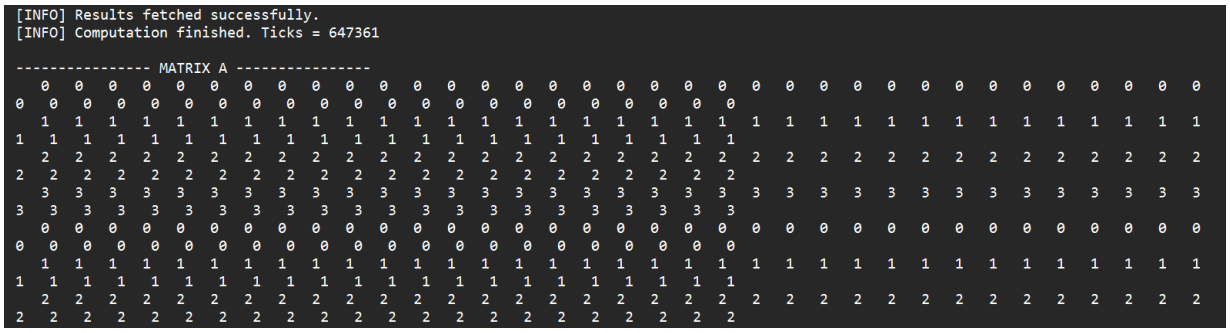

Figure 17: matrix A

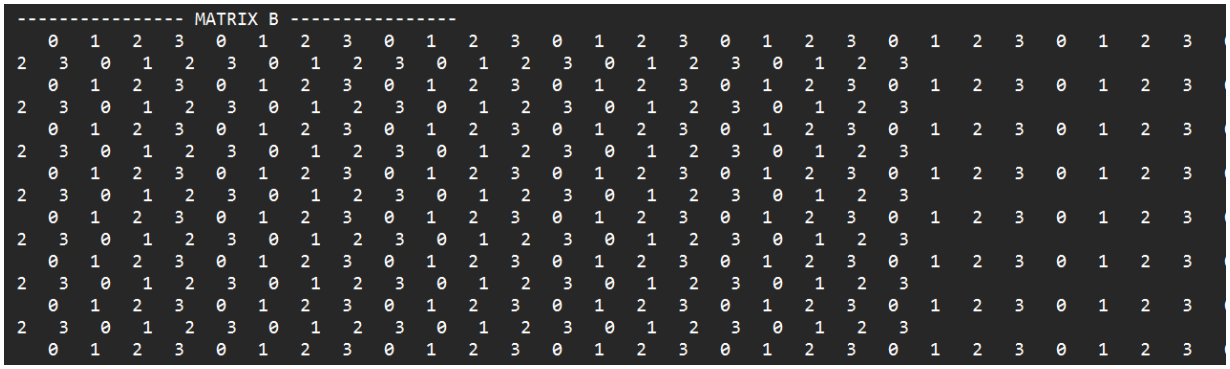

Figure 18: matrix B

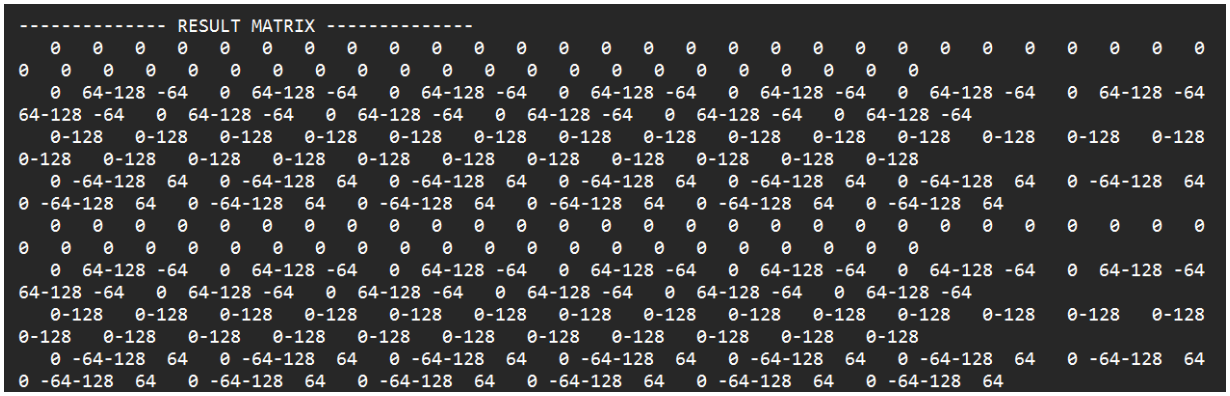

Figure 19: result matrix

Note: The computed values are clipped to 8-bit signed representation. Hence, overflow leads to wrapping, which results in negative values appearing in the output.

11.3 Test 3 : A x B

A matrix is such that $A[x][y] = x \ mod \ 4$ and $B[x][y] = x \ mod \ 4$

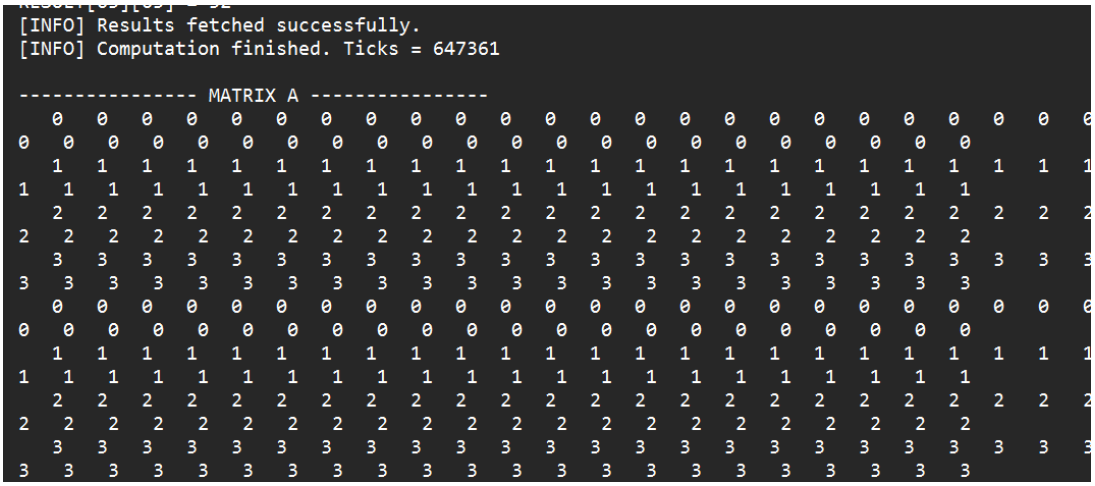

Figure 20: matrix A

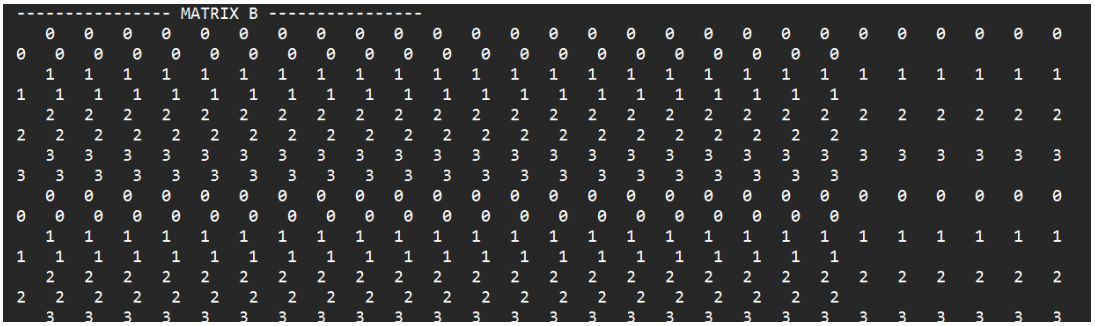

Figure 21: matrix B

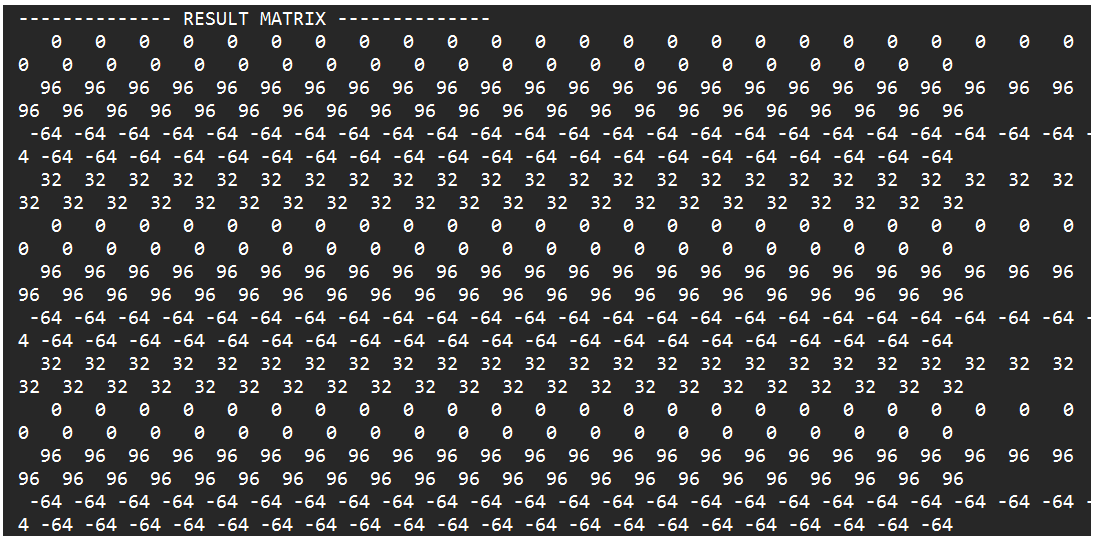

Figure 22: result matrix

12 Optimized Implementation 3 (Blocking and Tiling) : Working Screenshot

12.1 Test 1 : A x I

A matrix is such that $A[x][y] = x + y$ and I is identity matrix which results as output A

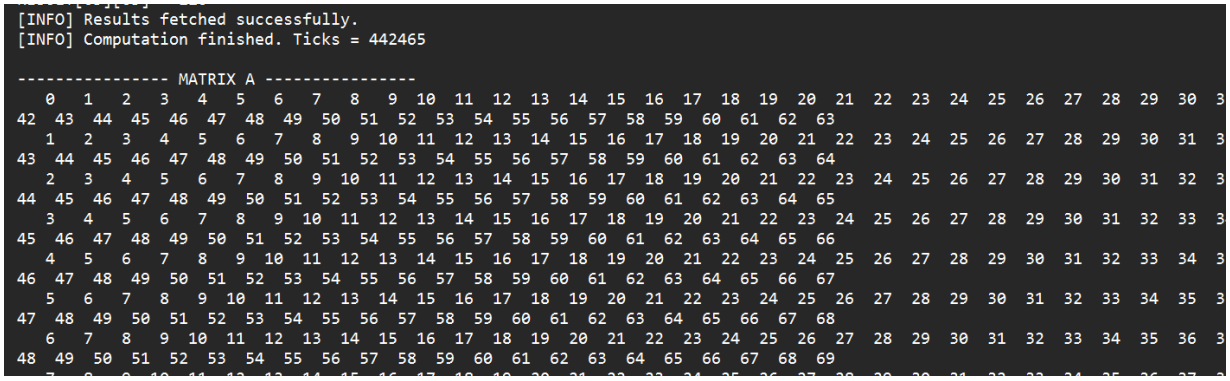

Figure 23: matrix A

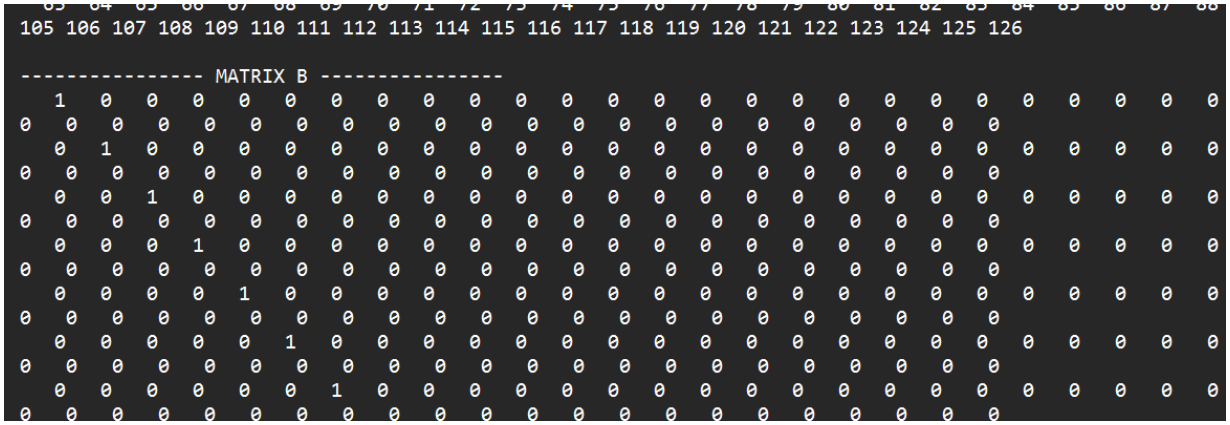

Figure 24: matrix B

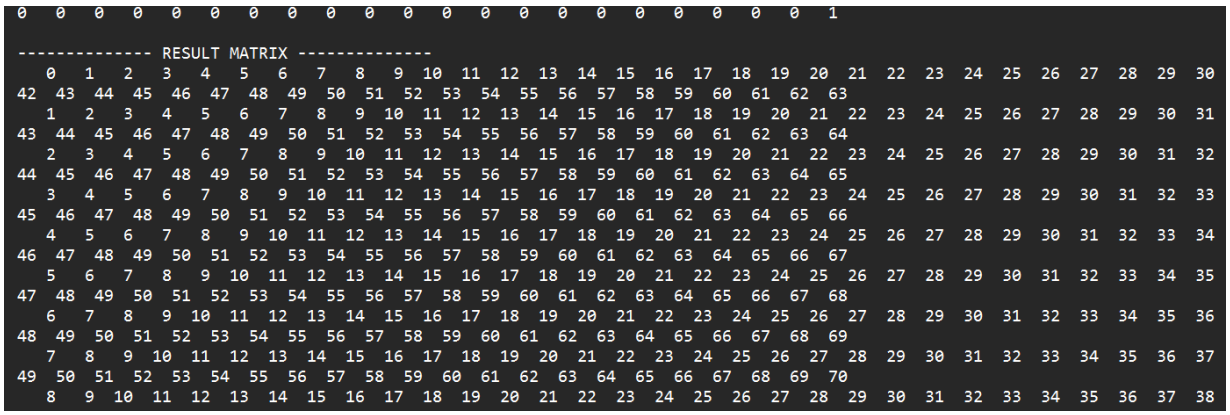

Figure 25: result matrix

12.2 Test 2 : A x B

A matrix is such that $A[x][y] = x \ mod \ 4$ and $B[x][y] = y \ mod \ 4$

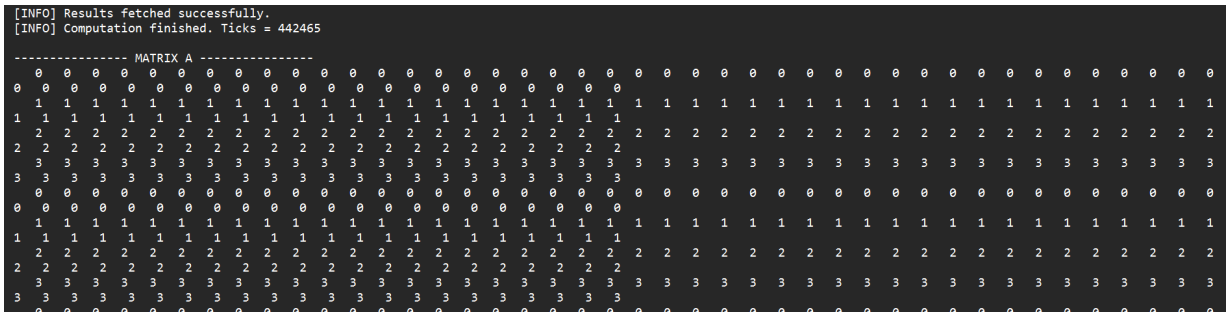

Figure 26: matrix A

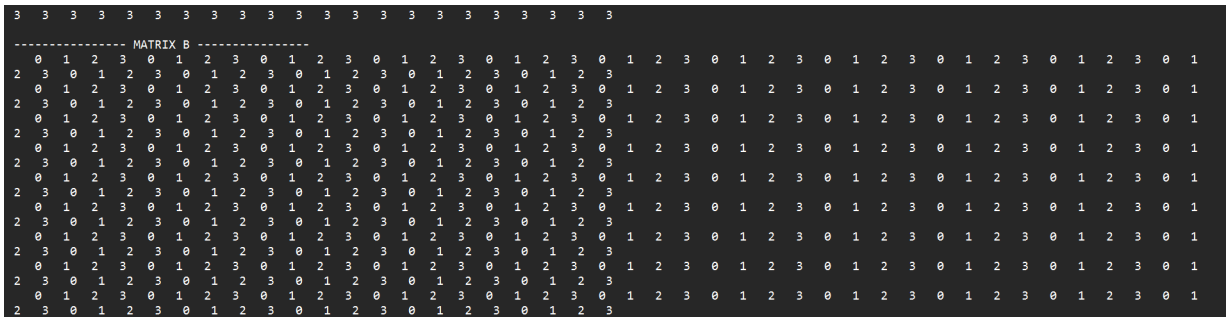

Figure 27: matrix B

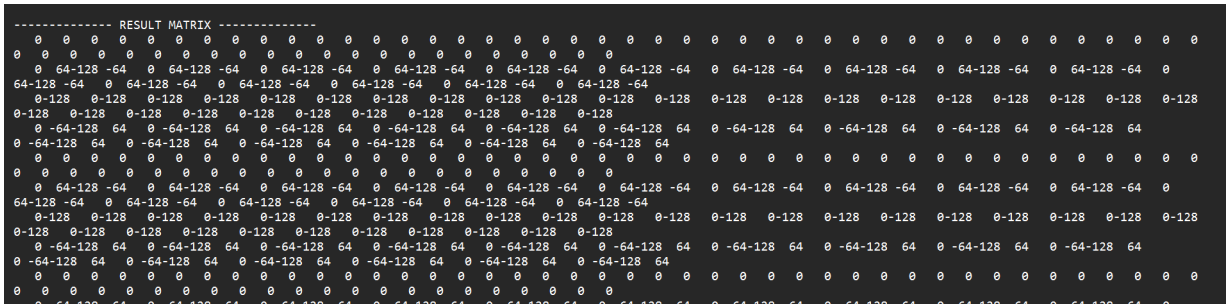

Figure 28: result matrix

Note: The computed values are clipped to 8-bit signed representation. Hence, overflow leads to wrapping, which results in negative values appearing in the output.

12.3 Test 3 : A x B

A matrix is such that $A[x][y] = x \ mod \ 4$ and $B[x][y] = x \ mod \ 4$

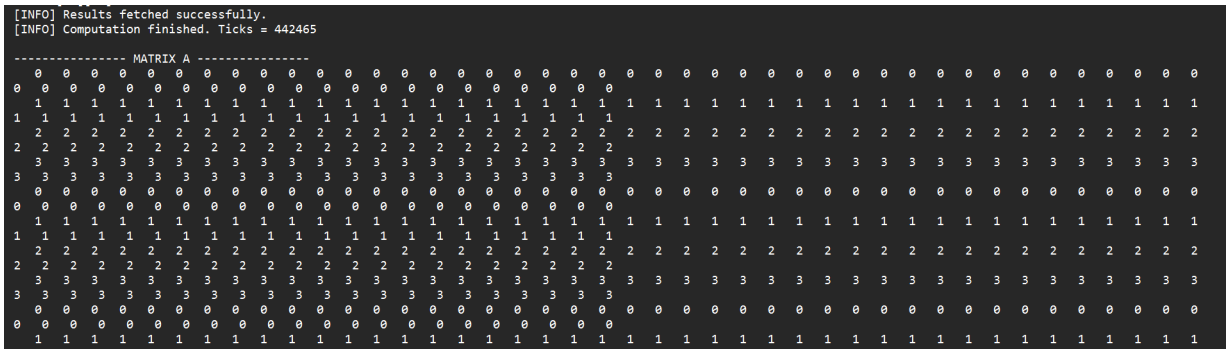

Figure 29: matrix A

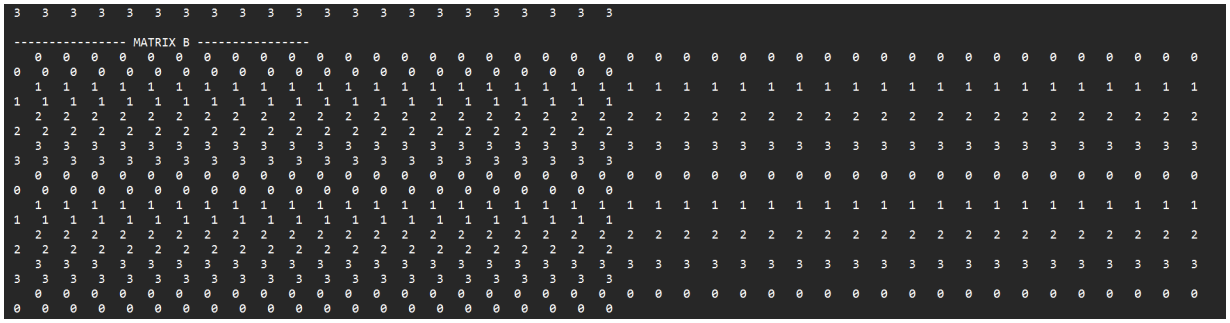

Figure 30: matrix B

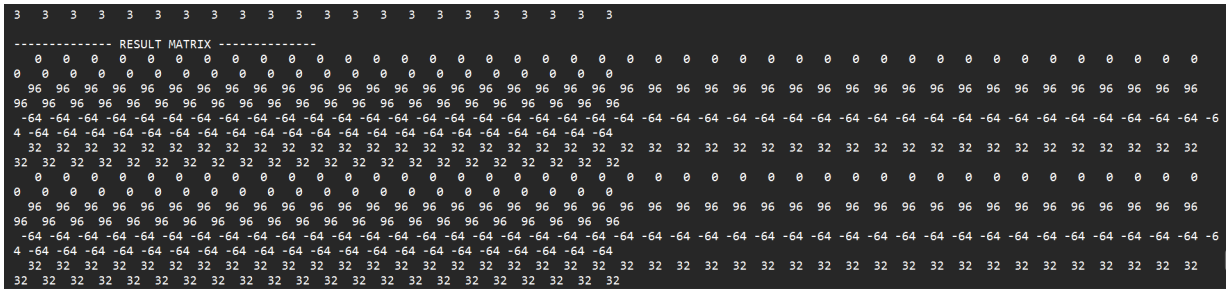

Figure 31: result matrix